

PFE — ISG BIZERTE / NOVATION CITY

DOCUMENTATION API

Maintenance Prédictive IA — Référence complète de l'API REST
Authentification, rate limiting, schémas de requêtes/réponses, 22 endpoints

Fichier source : `api_unified_pythagore.py`

Version API : 3.1.0 · Framework : FastAPI 0.115 + Uvicorn · Doc interactive : `/docs` , `/redoc`

Document généré le 07/08/2026

Table des matières

1. Vue d'ensemble
2. Authentification
3. Rate limiting
4. CORS
5. Modèles de données communs
6. Codes d'erreur
7. Endpoints — Système
8. Endpoints — IA / Prédiction
9. Endpoints — Reporting & Alertes
10. Endpoints — Pipeline (ré-entraînement)
11. Tableau récapitulatif

1. Vue d'ensemble

L'API **Maintenance Prédictive** expose 22 endpoints HTTP couvrant la détection d'anomalie sur roulements (ensemble de 6 modèles ML non supervisés), l'estimation de durée de vie résiduelle (RUL), l'analyse spectrale, le reporting et le pilotage du pipeline de ré-entraînement. Cette documentation couvre uniquement la surface HTTP de l'API — pour l'architecture globale du système (moteur temps réel, dashboards, entraînement), voir `DOCUMENTATION_TECHNIQUE.pdf`.

Caractéristique	Valeur
Base URL (local)	<code>http://localhost:8000</code>
Documentation interactive	<code>/docs</code> (Swagger UI) · <code>/redoc</code> (ReDoc)
Format des réponses	<code>application/json; charset=utf-8</code> (forcé explicitement)
Authentification	En-tête <code>X-API-Key</code> , 2 rôles (admin / opérateur)
Méthodes autorisées (CORS)	GET, POST
Nombre d'endpoints	22 (5 Système, 8 IA/Prédiction, 1 Reporting, 2 Alertes, 2 Données, 4 Pipeline)

Convention de lecture : chaque endpoint de ce document indique sa méthode HTTP, son chemin, son niveau d'authentification (**public** / **opérateur** ou **admin** / **admin uniquement**), sa limite de débit (rate limit), un exemple de requête/réponse et les codes d'erreur possibles.

2. Authentification

Implémentée dans `auth.py`. Système de clé API à 2 rôles (RBAC minimal), avec comparaison en **temps constant** (`hmac.compare_digest`) pour résister aux attaques temporelles.

2.1 En-tête requis

```
X-API-Key: <votre_clé>
```

Le nom de l'en-tête est configurable via la variable d'environnement `API_KEY_HEADER` (défaut : `X-API-Key`).

2.2 Rôles

Rôle	Variable d'environnement	Droits
admin	<code>API_KEYS=c1e1,c1e2</code>	Tous les endpoints protégés, y compris <code>/v1/pipeline/upload</code>
operator	<code>API_KEYS_OPERATOR=c1e3,c1e4</code>	Tous les endpoints protégés sauf <code>/v1/pipeline/upload</code>

Génération d'une clé sûre :

```
python -c "import secrets; print(secrets.token_hex(32))"
```

2.3 Comportement

Situation	Code	Message
En-tête absent	401	En-tête X-API-Key manquant.
Clé invalide	401	Clé API invalide.
Clé opérateur sur endpoint admin-only	403	Cette opération nécessite une clé API de niveau admin.
Aucune clé configurée côté serveur	503	Fail-closed — l'API refuse tout plutôt que de s'ouvrir par erreur

Chaque tentative (succès/échec) est journalisée dans `audit.log` (JSON Lines) avec horodatage, résultat (`GRANTED` / `DENIED` / `FORBIDDEN`), rôle, IP, méthode, chemin, et une **empreinte SHA-256 tronquée** de la clé — jamais la clé en clair.

2.4 Endpoints publics (sans clé)

`GET /`, `GET /health`, `/docs`, `/redoc`, `GET /metrics`, `GET /v1/model-card`, `GET /sensors`, `GET /anomalies`, `GET /v1/health-score/{id}`, `GET /v1/history/{id}`, `GET /v1/alert-level/{id}`, `GET /v1/results`, `GET /v1/spectral/{id}`, `GET /pipeline`, `GET /v1/alerts`, `GET /v1/alerts/stats`.

3. Rate limiting

Implémenté dans `rate_limiter.py` — fenêtre glissante en mémoire, thread-safe, sans dépendance externe (pas de Redis).

- **Clé de comptage** : `(IP source, chemin)` — pour les requêtes `POST`, le `sensor_id` extrait du corps JSON est ajouté à la clé (`ip:chemin:sensor_id`), pour que chaque capteur ait son propre quota plutôt qu'un quota partagé par IP sur `/v1/predict`.
- **Limite dépassée** → **429** avec en-tête `Retry-After: <secondes>`.
- **Purge automatique** des entrées inactives depuis plus d'1h (toutes les 5 min).
- La limite par minute varie selon l'endpoint (5 à 60 req/min) — voir le [tableau récapitulatif](#).

```
HTTP/1.1 429 Too Many Requests
Retry-After: 12
```

```
{"detail": "Trop de requetes. Limite : 60 appels/60s par IP. Reessayez dans 12s."}
```

Limite connue : basé sur `request.client.host`, qui reflète l'IP du reverse proxy si l'API est déployée derrière un proxy sans `proxy_headers=True` configuré côté Uvicorn — dans ce cas, tous les clients externes partageraient le même quota.

4. CORS

Contrôlé par la variable d'environnement `CORS_ORIGINS` :

Valeur	Effet
Non définie	* — toutes origines autorisées (adapté au développement)
Définie	Liste d'origines séparées par des virgules (à utiliser en production)

Méthodes autorisées : `GET`, `POST`. En-têtes : tous.

5. Modèles de données communs

5.1 `MeasurePoint` — une mesure capteur

Tous les champs sont optionnels sauf mention contraire par endpoint. Toute valeur hors plage déclenche une erreur de validation Pydantic (422).

Champ	Type	Plage physique	Description
<code>timestamp</code>	string (ISO 8601)	—	Horodatage de la mesure
<code>temperature</code>	float	-20 à 150 °C	Température
<code>vibration_x</code>	float	0 à 5000 mg	Vibration RMS axe X
<code>vibration_y</code>	float	0 à 5000 mg	Vibration RMS axe Y
<code>vibration_z</code>	float	0 à 5000 mg	Vibration RMS axe Z
<code>current</code>	float	0 à 500 A	Courant moteur
<code>power</code>	float	0 à 100 000 W	Puissance
<code>vitesse</code>	float	0 à 10 000 RPM	Vitesse
<code>acc_p2p</code>	float	0 à 30 000 mg	Accélération peak-to-peak (axe Y)
<code>acc_z2p</code>	float	0 à 15 000 mg	Accélération zero-to-peak (axe Y)
<code>acc_crest</code>	float	0 à 5000	Facteur de crête accélération
<code>acc_rms</code>	float	0 à 5000 mg	Accélération RMS (axe Y)

5.2 Dictionnaire `features`

Retourné par `/v1/predict` et `/v1/iot-predict`, calculé par `extract_features()` à partir de l'historique fourni (filtre médian anti-spike k=3 appliqué en amont sur température et vibration) :

Groupe	Clés
Thermique	<code>temp_mean</code> , <code>temp_std</code> , <code>temp_trend</code> , <code>temp_cur</code>
Vibration Z	<code>vib_z_mean</code> , <code>vib_z_std</code> , <code>vib_z_rms_w</code> , <code>vib_z_kurt</code> , <code>vib_z_crest</code> , <code>vib_z_cur</code>
Vibration X	<code>vib_x_mean</code> , <code>vib_x_std</code> , <code>vib_x_rms_w</code> , <code>vib_x_kurt</code>
Vibration Y	<code>vib_y_mean</code> , <code>vib_y_std</code> , <code>vib_y_rms_w</code> , <code>vib_y_kurt</code>
Ratios inter-axes	<code>vib_xy_ratio</code> , <code>vib_xz_ratio</code>
Courant	<code>current_mean</code> , <code>current_std</code> (toujours 0 — aucun capteur de courant installé)
Vibration totale (Pythagore 3D)	<code>vib_total</code> = $\sqrt{(X^2 + Y^2 + Z^2)}$ — base ISO 10816-3 / ISO 20816

Santé	health_score (0–100)
Accélération IFM	acc_p2p, acc_z2p, acc_crest, acc_rms (souvent 0 — limitation gateway)
V6 (avancées)	delta_vib, delta_temp, vib_entropy, fft_ratio, vib_asym_xy, vib_asym_xz

6. Codes d'erreur

Code	Signification	Exemple de cause
400	Requête invalide	<code>history</code> vide, moins de mesures que le minimum requis
401	Non authentifié	En-tête <code>X-API-Key</code> absent ou clé invalide
403	Authentifié mais non autorisé	Clé opérateur sur un endpoint admin-only
404	Ressource introuvable	Capteur sans historique, <code>job_id</code> inconnu
422	Erreur de validation Pydantic	Champ hors plage physique, type incorrect
429	Trop de requêtes	Rate limit dépassé (voir en-tête <code>Retry-After</code>)
500	Erreur serveur	Exception non gérée pendant le traitement
503	Service indisponible	Aucune clé API configurée, module optionnel non chargé (<code>signal_processing</code> , <code>reporting_module</code>)

7. Endpoints — Système

GET / — **public**

Identité de l'API et liste résumée des endpoints principaux. Pas de rate limit.

GET /health — **public**

Health check — état des modèles chargés. Pas de rate limit.

```
{
  "status": "ok",
  "models_loaded": true,
  "models": ["if", "lof", "ocsvm", "ecod", "hbos", "copod"],
  "features_count": 31,
  "version": "3.1.0",
  "n_sensors_in_memory": 19,
  "timestamp": "2026-08-07T10:00:00"
}
```

GET /metrics — **public** · 30/min

Métriques de performance du modèle (source : `models/metrics_v3.csv`) : `f1_score`, `accuracy`, `precision`, `recall`, `auc_roc`, `n_anomalies`, `n_total`, `contamination`, `weights` (poids appris du stacking par modèle), `window_size`, `n_features`.

GET /v1/model-card — **public** · 30/min

« Model card » — provenance des données d'entraînement, méthodologie, limites connues, pour chaque composant (détection d'anomalie et RUL). Permet d'auditer la fiabilité d'une prédiction avant utilisation pour une décision critique.

GET /v1/system-limits — **opérateur/admin** · 30/min

Liste structurée (L1 à L6) des limitations techniques connues : alertes non persistantes par défaut, features d'accélération toujours nulles, absence de capteur de courant, pas de déploiement edge, RUL heuristique non validé sur pannes réelles, etc.

8. Endpoints — IA / Prédiction

8.1 POST /v1/predict

POST /v1/predict — **opérateur/admin** · 60/min (par IP + sensor_id)

Détection d'anomalie en temps réel via un ensemble non supervisé **IF + LOF + OCSVM + ECOD + HBOS + COPOD**, fusionné par **stacking LogisticRegression**. Un filtre de **persistance temporelle** (k=3 fenêtres consécutives par défaut) confirme l'anomalie avant de la remonter, pour réduire les faux positifs isolés.

Requête (minimum 1 mesure dans `history`) :

```
{
  "sensor_id": "8f7f2f7e",
  "motor_id": "Motor_1604",
  "history": [
    { "temperature": 32.5, "vibration_x": 266.0, "vibration_y": 273.0,
      "vibration_z": 280.0, "current": 0 }
  ]
}
```

Réponse (200) :

```
{
  "sensor_id": "8f7f2f7e", "motor_id": "Motor_1604",
  "timestamp": "2026-08-07T10:00:00",
  "prediction": "NORMAL", "is_anomaly": false,
  "confidence": 0.12, "votes": 1, "risk_level": "FAIBLE",
  "anomaly_score": 0.12,
  "individual_models": { "IF": "NORMAL", "LOF": "ANOMALY", "OCSVM": "NORMAL",
    "ECOD": "NORMAL", "HBOS": "NORMAL", "COPOD": "NORMAL" },
  "individual_scores": { "IF": 0.08, "LOF": 0.61, "OCSVM": 0.15,
    "ECOD": 0.10, "HBOS": 0.09, "COPOD": 0.11 },
  "features": { "temp_mean": 32.5, "vib_total": 391.2, "health_score": 94.3, "...": "..." }
}
```

Champ	Détail
risk_level	FAIBLE (score<0.25) · MODÉRÉ (<0.50) · ÉLEVÉ (<0.75) · CRITIQUE (≥0.75) — renforcé à FAIBLE si health_score ≥85 sans anomalie confirmée
votes	Nombre de modèles (0–6) dont le score individuel dépasse 0.5

Effets de bord : met à jour l'historique glissant du capteur, alimente le buffer de vibration brute, déclenche une alerte externe si `risk_level` est CRITIQUE ou ÉLEVÉ.

Erreurs : 400 si `history` vide.

8.2 POST /v1/predict-rul

POST /v1/predict-rul — **opérateur/admin** · 60/min

Estime le temps restant avant défaillance (RUL), en combinant score de dégradation instantané, tendance temporelle, historique d'anomalies récentes, et un modèle ML (`GradientBoostingRegressor`) qui peut **aggraver** (jamais atténuer) le niveau d'alerte de l'heuristique. Minimum **3 mesures** dans `history`.

Le RUL est une **estimation empirique** — aucun modèle supervisé sur pannes réelles confirmées n'est disponible à ce jour. Ne pas utiliser comme seule base d'une décision d'arrêt machine.

Réponse (200) :

```
{
  "sensor_id": "8f7f2f7e", "motor_id": "Motor_1604",
  "timestamp": "2026-08-07T10:00:00",
  "rul_hours": 456.2, "rul_days": 19.01,
  "degradation_rate": 18.4, "health_score": 94.3,
  "confidence": "MOYENNE", "alert_level": "OK",
  "recommendation": "Fonctionnement normal. RUL > 14 jours...",
  "trend": { "temp_trend": 0.42, "vib_total_trend": 3.1,
    "deg_instant": 0.21, "deg_rate": 0.05,
    "rul_model": "heuristic_CDC + ML_gradient_boosting" }
}
```

Champ	Détail
<code>alert_level</code>	OK (>14j) · ATTENTION (7–14j) · URGENT (3–7j) · CRITIQUE (<3j)
<code>confidence</code>	HAUTE (≥10 mesures) · MOYENNE (≥5) · FAIBLE (<5)

Garde-fou capteur sain : `health_score` ≥85 force `alert_level=OK` avec un RUL plancher de 336h (14 jours).

Erreurs : 400 si moins de 3 mesures.

8.3 POST /v1/iot-predict

POST `/v1/iot-predict` — **opérateur/admin** · 60/min

Endpoint pensé pour la **production sans accès base de données**. Le client envoie **une mesure à la fois** ; le serveur maintient une fenêtre glissante de 20 mesures par `sensor_id` et retourne **prédiction d'anomalie + RUL en un seul appel** (RUL à `null` tant que moins de 3 mesures accumulées).

```
{
  "sensor_id": "8f7f2f7e", "temperature": 32.5,
  "vibration_x": 266.0, "vibration_y": 273.0, "vibration_z": 280.0
}
```

Champs requis : `temperature`, `vibration_x`, `vibration_y`, `vibration_z`. Réponse : union des champs de `/v1/predict` et `/v1/predict-rul`, plus `window_size`.

8.4 POST /v1/spectral-analysis

POST `/v1/spectral-analysis` — **opérateur/admin** · 20/min

Analyse spectrale complète du signal de vibration : FFT (spectre de puissance, fréquences dominantes), analyse d'enveloppe (démodulation Hilbert), fréquences caractéristiques de défauts roulements (**BPFO**, **BPFI**,

BSF, FTF — roulement SKF 6205-2RS), décomposition en ondelettes (CWT Morlet).

Requête identique à `/v1/predict`, plus paramètres query `rpm` (1450.0), `fs` (100.0 Hz). Minimum **8 mesures** `vibration_z`.

Erreurs : 400 si moins de 8 mesures ; 503 si `signal_processing.py` indisponible.

8.5 GET `/v1/spectral/{sensor_id}`

GET `/v1/spectral/{sensor_id}` — **public** · 30/min

Version publique et en lecture seule : utilise le buffer serveur des dernières valeurs `vibration_z` déjà reçues via `/v1/predict` (pas de recalcul côté client, pensé pour le dashboard). Paramètres query : `rpm`, `fs`.

8.6 GET `/v1/health-score/{sensor_id}`

GET `/v1/health-score/{sensor_id}` — **public** · 60/min

Score de santé (0–100) **normalisé par baseline propre au capteur** (calculée sur ses 20 premières mesures) — évite le biais d'une comparaison à un seuil global identique pour tous les moteurs.

```
{
  "sensor_id": "8f7f2f7e", "health_score": 91.4, "health_raw": 88.0,
  "baseline": 0.12, "score_method": "relatif_baseline",
  "anomaly_rate": 0.1, "n_records": 42, "last_score": 0.09,
  "trend": "STABLE", "timestamp": "2026-08-07T10:00:00"
}
```

Si le capteur n'a aucun historique : `health_score: 100.0`, `n_records: 0` (pas de 404 sur cet endpoint).

8.7 GET `/v1/history/{sensor_id}`

GET `/v1/history/{sensor_id}` — **public** · 60/min

Historique glissant des prédictions d'un capteur (en mémoire — réinitialisé au redémarrage, sauf persistance disque `anomaly_history_persist.json`). Paramètre query `limit` (défaut 20). Maximum en mémoire : 50 entrées par capteur.

Erreurs : 404 si aucun historique pour ce capteur.

8.8 GET `/v1/alert-level/{sensor_id}`

GET `/v1/alert-level/{sensor_id}` — **public** · 60/min

Niveau d'alerte **consolidé** sur les 10 dernières prédictions, pensé pour un feu tricolore de dashboard : `alert_level` OK (vert) · ATTENTION (jaune) · URGENT (orange) · CRITIQUE (rouge) · INCONNU (gris, si aucune donnée).

8.9 GET `/v1/results`

GET `/v1/results` — **public** · 60/min

Dernières prédictions (anomalie + RUL fusionnées) de tous les capteurs actifs, ou d'un capteur précis via `?sensor_id=`. Mis à jour à chaque appel `/v1/predict` ou `/v1/predict-rul`. Erreurs : 404 si `sensor_id` fourni mais introuvable.

8.10 GET /sensors et GET /anomalies

GET `/sensors` — **public** · 60/min

Liste des capteurs actifs (source : historique temps réel en mémoire, sinon CSV historique).

GET `/anomalies` — **public** · 60/min

Anomalies filtrées par score minimum. Paramètres query : `min_score` (0.5), `limit` (100).

9. Endpoints — Reporting & Alertes

9.1 GET /v1/report

GET /v1/report — **opérateur/admin** · 20/min

Génère un rapport de maintenance à partir des données temps réel.

Paramètre query	Valeurs
type	daily (24h, défaut) · weekly (7j) · monthly (30j) · full
format	json (défaut) · html
sensor_id	optionnel — filtre sur un seul capteur

Erreurs : 400 si type invalide ; 503 si reporting_module.py indisponible.

9.2 GET /v1/alerts

GET /v1/alerts — **public** · 30/min

Historique des alertes externes envoyées (email/webhook/SMS) par AlertManager, avec statut de livraison par canal (booléens uniquement — jamais d'adresse, URL ou identifiant exposé). Paramètre query limit (défaut 50).

9.3 GET /v1/alerts/stats

GET /v1/alerts/stats — **public** · 30/min

Statistiques globales du gestionnaire d'alertes : total envoyées, répartition par niveau, cooldowns actifs.

Note de cohérence documentaire : /v1/alerts et /v1/alerts/stats ne portent pas de dépendance require_api_key dans le code actuel de api_unified_pythagore.py — ils sont donc accessibles sans clé, comme les autres endpoints de monitoring en lecture seule.

10. Endpoints — Pipeline (ré-entraînement)

Workflow complet de ré-entraînement à partir d'un dump SQL MariaDB (`ai_cp`), exécuté en arrière-plan (thread) : parsing SQL → CSV, entraînement des 6 modèles, rechargement à chaud dans l'API (sans redémarrage).

10.1 GET /pipeline

GET /pipeline — **public**

Sert la page HTML d'upload (`pipeline_upload.html`). Exclue du schéma OpenAPI.

10.2 POST /v1/pipeline/upload

POST /v1/pipeline/upload — **admin uniquement** · 5/min

Reçoit un fichier `.sql` (flux par blocs de 4 Mo, pour éviter de charger un dump de plusieurs centaines de Mo entièrement en mémoire), puis démarre en tâche de fond : parsing SQL (`generate_dataset_from_sql.py`), entraînement (`train_model_v3_unsupervised.py`), rechargement des modèles.

Requête : `multipart/form-data` — `file` (.sql, requis), `train_mode` ("full" ou "fast", défaut "full").

```
curl -X POST http://localhost:8000/v1/pipeline/upload \
-H "X-API-Key: <clé_admin>" \
-F "file=@dump_ai_cp.sql" \
-F "train_mode=full"
```

```
{
  "job_id": "a1b2c3d4e5f6", "status": "running",
  "filename": "dump_ai_cp.sql", "size_mb": 87.4,
  "poll_url": "/v1/pipeline/status/a1b2c3d4e5f6"
}
```

Erreurs : 400 si l'extension n'est pas `.sql` ; 401/403 selon le rôle de la clé — **une clé opérateur valide reçoit 403, pas 401.**

10.3 GET /v1/pipeline/status/{job_id}

GET /v1/pipeline/status/{job_id} — **opérateur/admin** · 60/min

Statut d'un job en cours — à interroger toutes les 2-3 secondes. Paramètre query `since` (défaut 0) : ne retourne que les logs postérieurs à cet index.


```
{
  "job_id": "alb2c3d4e5f6", "status": "running", "step": 4,
  "step_name": "Entraînement des modèles ML...", "progress": 58,
  "elapsed": "1m 12s",
  "new_logs": [{"text": "✅ LOF entraîné", "type": "s"}],
  "total_logs": 34, "results": null
}
```

`status` : `running` · `done` · `error`. `results` (non nul une fois terminé) inclut `auc`, `f1`, `n_measures`, `n_anomalies`, `top_anomalies`.

Erreurs : 404 si `job_id` inconnu.

10.4 GET /v1/pipeline/jobs

GET /v1/pipeline/jobs — **opérateur/admin** · 30/min

Liste des 10 derniers jobs de pipeline (résumé : `job_id`, `status`, `filename`, `progress`, `elapsed`, `created_at`).

11. Tableau récapitulatif

Méthode	Endpoint	Auth	Rate limit (req/min)	Tag
GET	/	Non	—	Système
GET	/health	Non	—	Système
GET	/metrics	Non	30	Système
GET	/v1/model-card	Non	30	Système
GET	/v1/system-limits	Oui	30	Système
POST	/v1/predict	Oui	60	IA / Prédiction
POST	/v1/predict-rul	Oui	60	IA / Prédiction
POST	/v1/iot-predict	Oui	60	IA / Prédiction
POST	/v1/spectral-analysis	Oui	20	IA / Prédiction
GET	/v1/spectral/{sensor_id}	Non	30	IA / Prédiction
GET	/v1/health-score/{sensor_id}	Non	60	IA / Prédiction
GET	/v1/history/{sensor_id}	Non	60	IA / Prédiction
GET	/v1/alert-level/{sensor_id}	Non	60	IA / Prédiction
GET	/v1/results	Non	60	IA / Prédiction
GET	/sensors	Non	60	Données
GET	/anomalies	Non	60	Données
GET	/v1/report	Oui	20	Reporting
GET	/v1/alerts	Non	30	Alertes
GET	/v1/alerts/stats	Non	30	Alertes
GET	/pipeline	Non	—	Pipeline
POST	/v1/pipeline/upload	Admin uniquement	5	Pipeline
GET	/v1/pipeline/status/{job_id}	Oui	60	Pipeline
GET	/v1/pipeline/jobs	Oui	30	Pipeline

Source : `api_unified_pythagore.py`, `auth.py`, `rate_limiter.py`. Document généré à partir de `API_DOCUMENTATION.md`.